

IF2224 FORMAL LANGUAGE AND AUTOMATA THEORY

Arion Interpreter — Milestone 1: Lexical Analysis

April 1, 2026

Made Branenda Jordhy — 13524026

Muhammad Nur Majiid — 13524028

Jason Edward Salim — 13524034

Athilla Zaidan Zidna Fann — 13524068

**School of Electrical Engineering and Informatics
Institut Teknologi Bandung
Semester II 2025/2026**

Contents

1	Theoretical Background	3
1.1	Lexical Analysis	3
1.2	Token	3
1.3	Deterministic Finite Automaton (DFA)	3
2	Design and Implementation	4
2.1	Overview	4
2.2	DFA Design	4
2.3	Tokenization Flow	7
2.4	Token Classification	7
3	Testing	9
3.1	Lokasi File Pengujian	9
3.2	Hasil Uji Berdasarkan Screenshot	10
4	Conclusion	14
	Appendix	16

1 Theoretical Background

1.1 Lexical Analysis

Lexical analysis adalah fase pertama dalam pipeline kompilasi maupun interpretasi. Fase ini bertanggung jawab mengubah aliran karakter mentah dari source code menjadi aliran *token* berstruktur yang siap dikonsumsi oleh parser. Komponen yang melaksanakan proses ini disebut *lexer* atau *scanner*.

Lexer beroperasi dengan membaca input secara sekuensial, satu karakter per iterasi, sambil memelihara state internal yang merepresentasikan pola yang sedang dikenali. Karakter yang tidak membawa makna semantis, yaitu whitespace dan komentar, dibuang pada fase ini sehingga tidak membebani tahap analisis selanjutnya. Keluaran lexer adalah barisan token yang menjadi representasi leksikal dari program, dan barisan inilah yang diserahkan ke *syntax analyzer* untuk diproses lebih lanjut.

1.2 Token

Sebuah *token* adalah pasangan (t, v) di mana t menyatakan *token type* dan v menyatakan *lexeme*, yaitu substring dari source code yang membentuk token tersebut. Sebagai contoh, ekspresi `x := 5` menghasilkan tiga token: `IDENT("x")`, `BECOMES`, dan `INTCON("5")`.

Token dalam bahasa pemrograman umumnya terklasifikasi ke dalam kategori berikut.

- **Literals.** Konstanta bernilai tetap: integer, bilangan riil, karakter, dan string.
- **Keywords.** Reserved words yang memiliki semantik khusus dalam bahasa, seperti `if`, `while`, `begin`, dan `function`.
- **Identifiers.** Nama yang didefinisikan pengguna untuk variabel, fungsi, prosedur, dan tipe.
- **Operators.** Simbol operasi aritmatika, logika, dan relasional.
- **Delimiters.** Karakter pembatas struktural seperti `(`, `)`, `;`, dan `..`

Perlu diperhatikan bahwa keyword dan identifier secara leksikal memiliki pola reguler yang identik. Disambiguasi dilakukan setelah lexeme terbaca secara lengkap: lexeme yang terdaftar sebagai reserved word dikategorikan sebagai keyword, selainnya sebagai identifier.

1.3 Deterministic Finite Automaton (DFA)

Pola leksikal dari setiap kategori token dapat dideskripsikan oleh *regular expression*, dan setiap regular expression ekuivalen dengan sebuah *finite automaton*. Model komputasi yang digunakan dalam implementasi lexer adalah *Deterministic Finite Automaton* (DFA).

Definisi. Sebuah DFA adalah 5-tupel $M = (Q, \Sigma, \delta, q_0, F)$, di mana: Q adalah himpunan state berhingga, Σ adalah alfabet input, $\delta : Q \times \Sigma \rightarrow Q$ adalah fungsi transisi total, $q_0 \in Q$ adalah state awal, dan $F \subseteq Q$ adalah himpunan accepting states.

Sifat deterministik dari DFA dinyatakan oleh totalitas δ : untuk setiap $(q, a) \in Q \times \Sigma$, terdapat tepat satu state $q' = \delta(q, a)$. Hal ini menjamin bahwa eksekusi DFA pada string input manapun bersifat unik dan dapat diprediksi, tanpa backtracking maupun pencabangan komputasi.

Dalam penggunaan sebagai lexer, DFA beroperasi sebagai berikut. Dimulai dari q_0 , setiap karakter a_i dari input menginduksi transisi $\delta(q_{\text{curr}}, a_i) = q_{\text{next}}$. Apabila $q_{\text{curr}} \in F$ dan $\delta(q_{\text{curr}}, a_{i+1})$ tidak terdefinisi atau mengarah ke state error, maka lexeme yang terakumulasi diemisikan sebagai token. Automaton kemudian direset ke q_0 untuk memulai pengenalan token berikutnya. Proses ini berjalan dalam waktu $O(n)$ terhadap panjang input n , karena setiap karakter diproses dalam satu transisi $O(1)$.

2 Design and Implementation

2.1 Overview

Program lexer diimplementasikan dalam C++17 menggunakan pendekatan DFA berbasis pembacaan karakter tunggal. Setiap token dikenali melalui satu karakter per iterasi, di mana state machine memutuskan transisi berdasarkan `currChar` dan, bila diperlukan, satu karakter *lookahead* melalui `f.peek()`. Penggunaan lookahead tunggal tetap konsisten dengan model DFA karena setiap transisi tetap deterministik: tidak ada backtracking dan tidak ada pilihan ganda.

Program terdiri dari tiga komponen yang terpisah: `token.hpp` mendefinisikan enum `ArionToken` dan kelas `Token`; `lexer.hpp/lexer.cpp` mengimplementasikan kelas `Lexer` yang menjalankan DFA; dan `main.cpp` menangani antarmuka I/O serta pemrosesan berkas.

2.2 DFA Design

DFA yang diimplementasikan memiliki state awal q_0 yang berfungsi sebagai titik reset setelah setiap token diemisikan. Dari q_0 , karakter pertama yang terbaca menentukan cabang sub-automaton yang akan dimasuki. Diagram 1 menampilkan gambaran umum transisi dari q_0 ke seluruh state utama. Untuk keterbacaan, token berkarakter tunggal (+, -, *, /, (,), [,], ,, ;, .) masing-masing memiliki transisi langsung dari q_0 ke sebuah *accepting state* dan tidak ditampilkan secara individual pada diagram.

Beberapa pola pengenalan memerlukan perhatian lebih lanjut. Diagram 2 menampilkan detail transisi untuk sub-automaton yang lebih kompleks.

Catatan mengenai keyword. Keyword dan identifier memiliki pola leksikal yang identik: keduanya dimulai dengan huruf dan dapat diikuti kombinasi huruf dan angka. Disambiguasi tidak dilakukan melalui state DFA yang terpisah, melainkan melalui *reserved word lookup* pada `std::unordered_map` setelah seluruh lexeme terbaca oleh q_w . Pendekatan ini ekuivalen secara teoritis karena tabel lookup adalah fungsi total dari string ke kategori token, tanpa memerlukan pencabangan automaton. Selain itu, nilai *lexeme* dipertahankan dalam huruf asli (case-sensitive) untuk identifier, sedangkan keyword tidak menyimpan nilai lexeme.

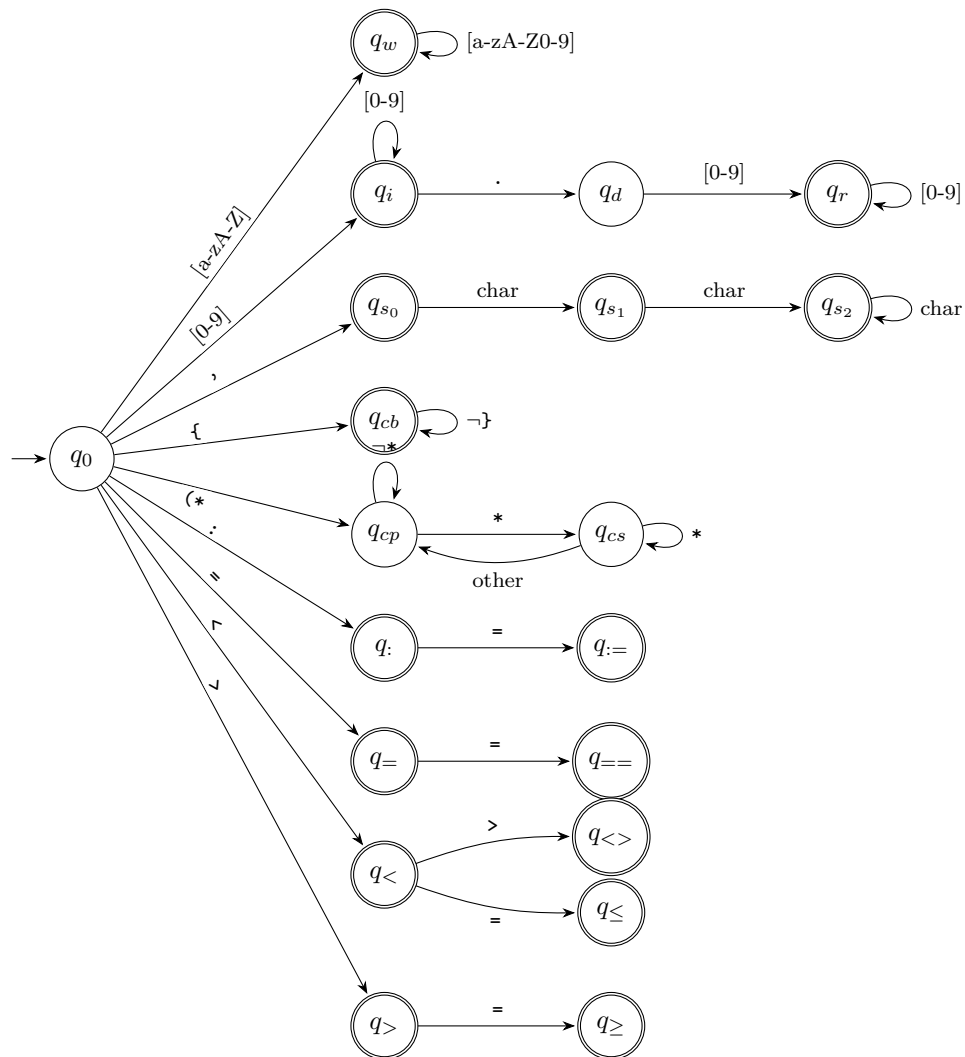
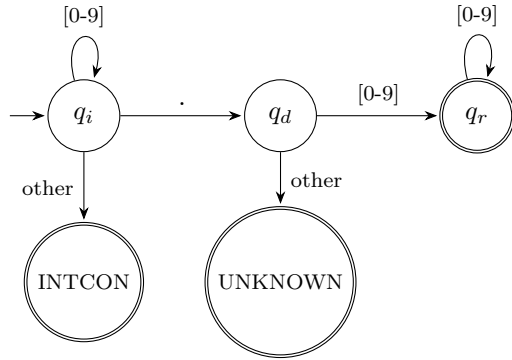
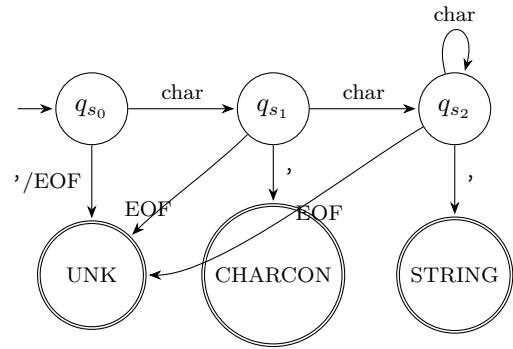


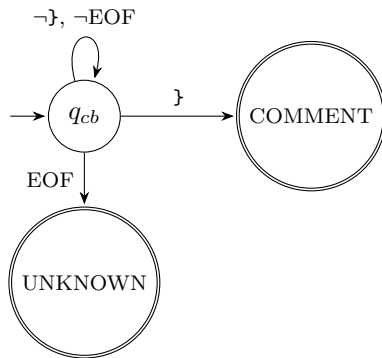
Figure 1: DFA overview: state awal q_0 dan seluruh state utama. State dengan lingkaran ganda adalah *accepting states*. Token berkarakter tunggal tidak ditampilkan individual.



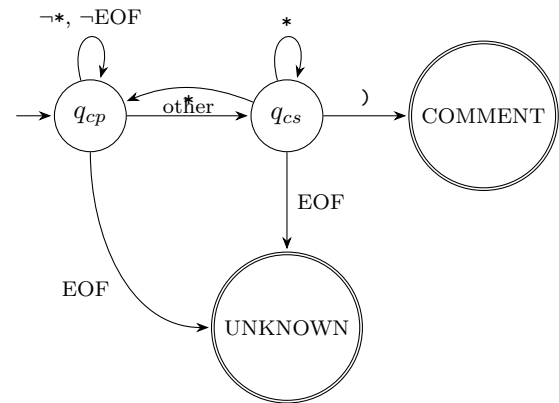
(a) Number recognition



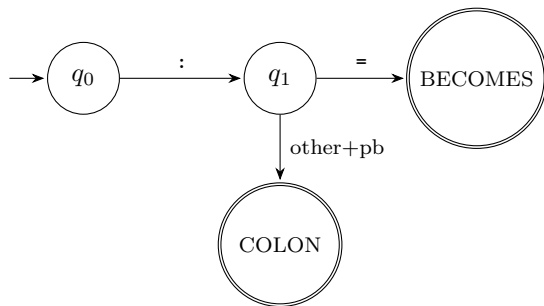
(b) String/char recognition



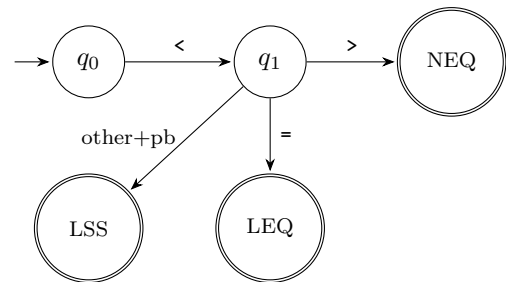
(c) Comment { ... } recognition



(d) Comment (*...*) recognition



(e) Colon / Becomes recognition



(f) Relational < operators recognition

Figure 2: Detail sub-automaton untuk pola leksikal kompleks.

2.3 Tokenization Flow

Gambar 3 menunjukkan alur eksekusi fungsi `tokenize()`. Pada setiap iterasi, whitespace dilewati terlebih dahulu, kemudian karakter saat ini digunakan sebagai kondisi dispatch ke sub-automaton yang sesuai. Apabila karakter tidak dikenali oleh pola manapun, token `UNKNOWN` diemisikan sebagai penanda kesalahan leksikal.

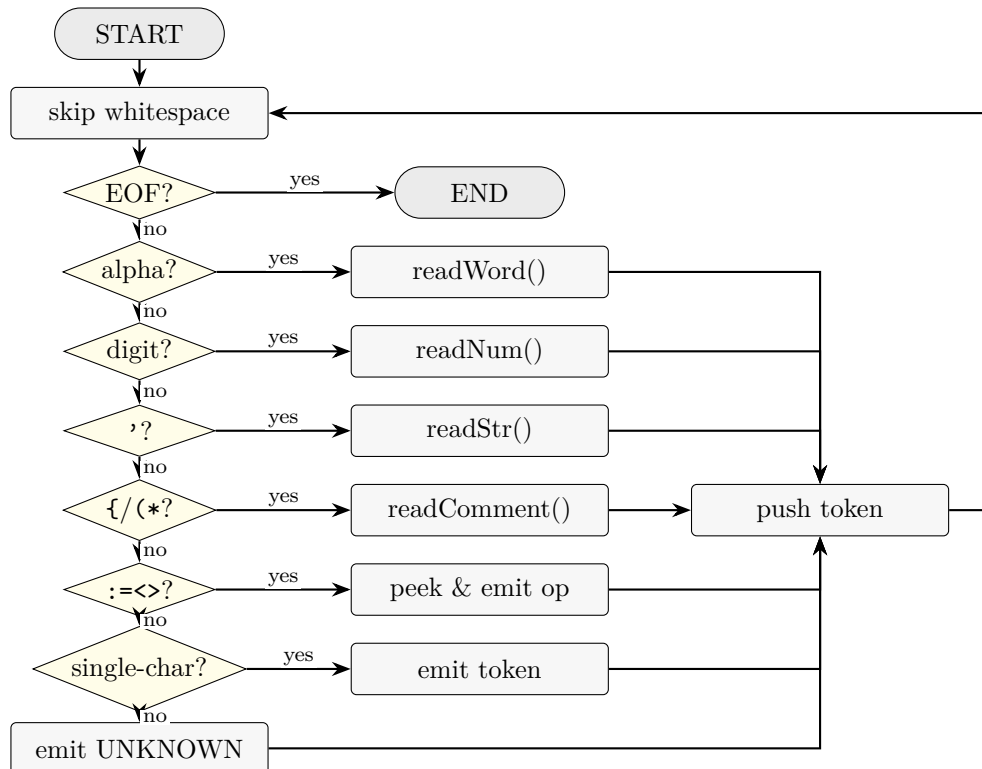


Figure 3: Alur eksekusi fungsi `tokenize()`.

2.4 Token Classification

Tabel 1 merangkum seluruh 53 jenis token yang dikenali oleh lexer beserta pola atau simbol yang merepresentasikannya, dikelompokkan berdasarkan kategori.

Table 1: Daftar token bahasa Arion.

Category	Token	Pattern / Example
<i>Literals</i>		
	<code>intcon</code>	Integer constant: 1, 42, 100
	<code>realcon</code>	Real constant: 3.14, 0.5
	<code>charcon</code>	Single char in quotes: 'A', 'z'
	<code>string</code>	Multi-char in quotes: 'hello', 'IRK'
<i>Arithmetic Operators</i>		
	<code>plus</code>	+
	<code>minus</code>	-
	<code>times</code>	*
	<code>rdiv</code>	/

Continued on next page...

(continued)

Category	Token	Pattern / Example
	idiv	div
	imod	mod
<i>Logical Operators</i>		
	andsy	AND
	orsy	OR
	notsy	NOT
<i>Relational Operators</i>		
	eql	==
	neq	<>
	lss	<
	leq	<=
	gtr	>
	geq	>=
<i>Delimiters & Punctuation</i>		
	lparent	(
	rparent	)
	lbrack	[
	rbrack	]
	comma	,
	semicolon	;
	period	.
	colon	:
	becomes	:=
<i>Declaration Keywords</i>		
	constsy	const
	typesy	type
	varsy	var
	functionsy	function
	proceduresy	procedure
	arraysy	array
	recordsy	record
	programsy	program
<i>Control Flow Keywords</i>		
	beginsy	begin
	endsy	end
	ifsy	if
	thensy	then
	elsesy	else
	casesy	case
	ofsy	of
	whilesy	while
	dosy	do
	repeatsy	repeat
	untilsy	until
	forsy	for
	tosy	to

Continued on next page...

(continued)

Category	Token	Pattern / Example
	downtosy	downto
<i>Special</i>		
	ident	Identifier: <code>x</code> , <code>myVar</code> , <code>h4g</code>
	comment	<code>{...}</code> or <code>(*...*)</code> (discarded)
	unknown	Unrecognized token (lexical error)

3 Testing

Pengujian(testing) Program dilakukan menggunakan testcase yang sudah dibuat dan disimpan pada direktori: `test/milestone-1/`. Setiap testcase memiliki pasangan berkas input-output dengan format nama: `input-{n}.txt` dan `output-{n}.txt`.

3.1 Lokasi File Pengujian

- Berkas input testing: `test/milestone-1/input-1.txt` s.d. `test/milestone-1/input-6.txt`
- Berkas output testing: `test/milestone-1/output-1.txt` s.d. `test/milestone-1/output-6.txt`

3.2 Hasil Uji Berdasarkan Screenshot

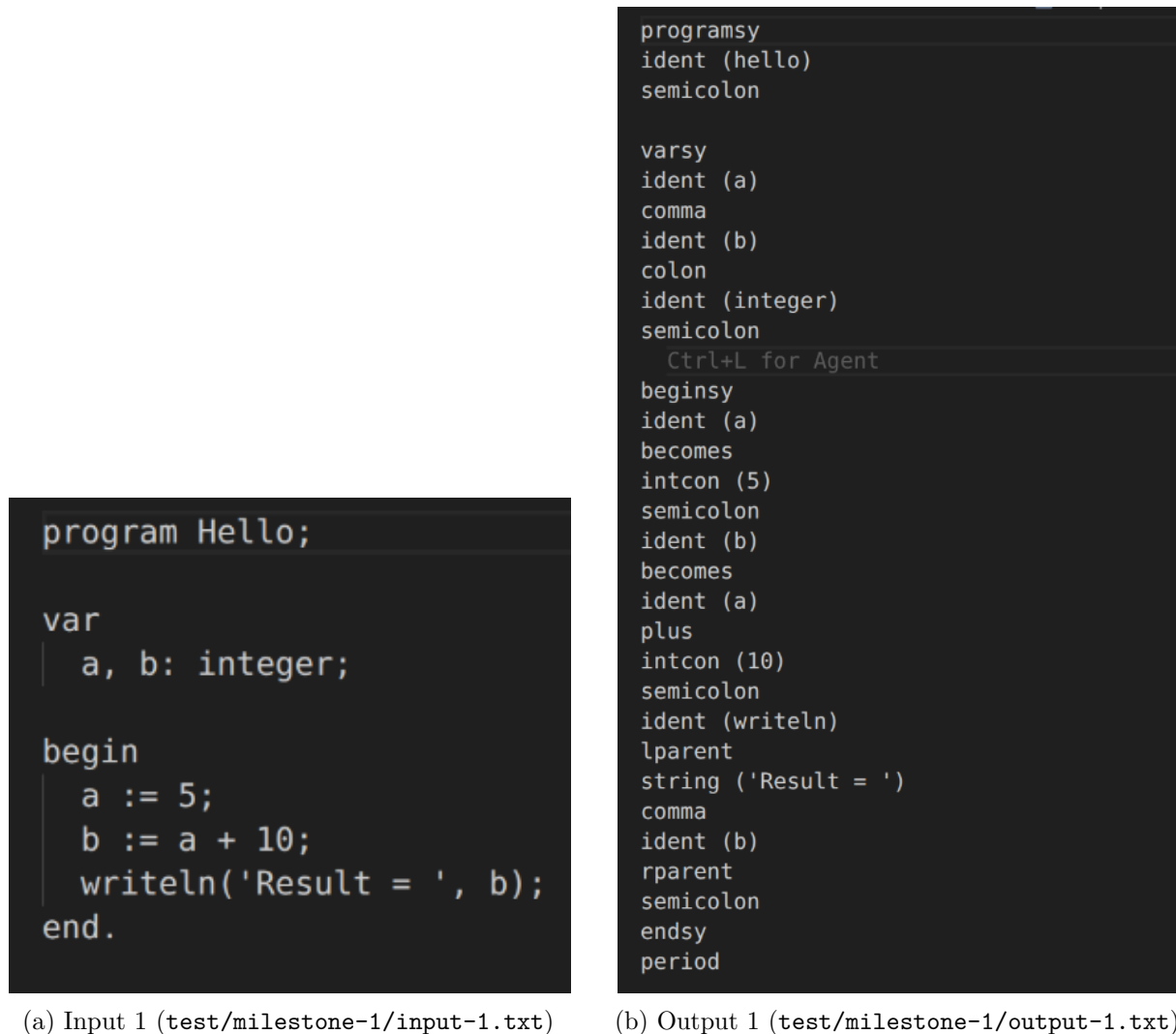


Figure 4: Pasangan hasil uji kasus 1.

```
program TestError;

var
  x: integer;
  y: real;

begin
  x := 10;
  y := 3.14;

  if x == y then
    writeln('sama');

  x := @;

  writeln(1.);

  writeln('belum ditutup);
end.
```

Ctrl+L for Agent

(a) Input 3 (test/milestone-1/input-3.txt)

```
programsy
ident (testerror)
semicolon

varsy
ident (x)
colon
ident (integer)
semicolon
ident (y)
colon
ident (real)
semicolon

beginsy
ident (x)
becomes
intcon (10)
semicolon
ident (y)
becomes
realcon (3.14)
semicolon

ifsy
ident (x)
eql
ident (y)
thensy
ident (writeln)
lparent
string ('sama')
rparent
semicolon

ident (x)
becomes
unknown (@)
semicolon

ident (writeln)
lparent
unknown (1.)
rparent
semicolon

ident (writeln)
lparent
string ('belum ditutup);
end.
```

(b) Output 3 (test/milestone-1/output-3.txt)

Figure 5: Pasangan hasil uji kasus 3 (mengandung lexical error).

```
program Operator;  
  
var  
  a, b: integer;  
  
begin  
  a := 10;  
  b := 20;  
  
  if a <> b then  
    writeln('tidak sama');  
  
  if a <= b then  
    writeln('a kecil');  
  
  if b >= a then  
    writeln('b besar');  
end.
```

(a) Input 4 (test/milestone-1/input-4.txt)

```
programsy  
ident (operator)  
semicolon  
  
varsy  
ident (a)  
comma  
ident (b)  
colon  
ident (integer)  
semicolon  
  
beginsy  
ident (a)  
becomes  
intcon (10)  
semicolon  
ident (b)  
becomes  
intcon (20)  
semicolon  
  
ifsy  
ident (a)  
neq  
ident (b)  
thensy  
ident (writeln)  
lparent  
string ('tidak sama')  
rparent  
semicolon  
  
ifsy  
ident (a)  
leq  
ident (b)  
thensy  
ident (writeln)  
lparent  
string ('a kecil')  
rparent  
semicolon  
  
ifsy  
ident (b)  
geq  
ident (a)  
thensy  
ident (writeln)  
lparent  
string ('b besar')  
rparent  
semicolon  
endsy  
period
```

(b) Output 4 (test/milestone-1/output-4.txt)

Figure 6: Pasangan hasil uji kasus 4 (operator relasional).

```

program Rekord;

type
  Titik = record
    x, y: integer;
  end;

var
  p: Titik;

begin
  p.x := 3;
  p.y := 4;
  writeln('Jarak: ', p.x + p.y);
end.

```

(a) Input 5 (test/milestone-1/input-5.txt)

```

programsy
ident (rekord)
semicolon

typesy
ident (titik)
unknown (=)
recordsy
ident (x)
comma
ident (y)
colon
ident (integer)
semicolon
endsy
semicolon

varsy
ident (p)
colon
ident (titik)
semicolon

beginsy
ident (p)
period
ident (x)
becomes
intcon (3)
semicolon
ident (p)
period
ident (y)
becomes
intcon (4)
semicolon
ident (writeln)
lparent
string ('Jarak: ')
comma
ident (p)
period
ident (x)
plus
ident (p)
period
ident (y)
rparent
semicolon
endsy
period

```

(b) Output 5 (test/milestone-1/output-5.txt)

Figure 7: Pasangan hasil uji kasus 5 (fitur **record** dan akses field).

```
program CampuranError;

var
  n: integer;

begin
  n := 5;

  { komentar valid }

  (* komentar tidak ditutup

  n := n + #;

  writeln('');

  if n = 10 then
    writeln('ok');
  end.

programsy
ident (campuranerror)
semicolon

varsy
ident (n)
colon
ident (integer)
semicolon

beginsy
ident (n)
becomes
intcon (5)
semicolon

unknown (unclosed comment)
```

(a) Input 6 (test/milestone-1/input-6.txt)

(b) Output 6 (test/milestone-1/output-6.txt)

Figure 8: Pasangan hasil uji kasus 6 (unclosed comment).

4 Conclusion

Dari pengerjaan Milestone 1 ini, komponen *Lexical Analyzer* (Lexer) untuk Arion Interpreter berhasil diimplementasikan dan diuji. Pembuatan lexer ini sepenuhnya menggunakan bahasa pemrograman C++17, dengan menerapkan *Deterministic Finite Automaton* (DFA) yang dirancang untuk membaca *source code* karakter demi karakter. Secara keseluruhan, program lexer sudah sanggup berjalan dengan baik sesuai spesifikasi bahasa yang diharapkan.

Salah satu poin penting dari arsitektur lexer ini adalah proses pemindaianya yang terbukti tidak membutuhkan *backtracking*. Semata-mata karena memanfaatkan sistem satu karakter *lookahead*, transisi antar *state* di dalam DFA bisa dieksekusi secara deterministik. Hal ini berdampak bagus pada kinerjanya yang efisien dengan kompleksitas waktu yang linier, yakni $O(n)$ terhadap panjang input. Alhasil, proses ini relatif lebih hemat memori dan aman saat dieksekusi.

Program lexer ini sukses mengidentifikasi dan mengelompokkan 53 jenis tipe token dasar yang ada di dalam spesifikasi bahasa Arion. Proses klasifikasinya juga sudah mencakup unsur pengenalan seperti *literals*, *operators*, *delimiters*, hingga *keywords* dan *identifiers*. Selain itu, lexer terbukti pintar mendeteksi serta membuang karakter berlebih yang tidak ada fungsinya (non-semantis), sebut saja semua variasi jarak pemisah kata (*whitespace*) serta dua macam tata cara format penulisan komentar.

Kesimpulannya, keluaran langsung dari pengerjaan leksikal ini terbukti sukses mencetak *token stream* yang rapi, utuh, dan terstruktur. Karena keluaran barisan token ini sudah dipilah-pilah dari gangguan sinyal ("derau") dan tidak punya sisi ketidakpastian alias ambigu, ini bakal mempermudah kita mengerjakan langkah lanjut di tahapan Milestone 2. Intinya, landasan token ini sudah siap digunakan untuk pekerjaan membangun representasi *Abstract Syntax Tree* (AST) pada pengerjaan *Syntax Analysis* / Parsing selanjutnya.

References

- [1] “Compiler Design - Lexical Analysis,” TutorialsPoint. https://www.tutorialspoint.com/compiler_design/compiler_design_lexical_analysis.htm
- [2] J. E. Hopcroft, R. Motwani, and J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed. Pearson, 2006.
- [3] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Addison-Wesley, 2006.
- [4] “An Introduction to Makefiles,” GNU Make Manual. https://www.gnu.org/software/make/manual/html_node/Introduction.html

Appendix

GitHub Repository

<https://github.com/jsndwrld/DCL-Tubes-IF2224-2026>

DFA Transition Diagram

Task Distribution

NIM	Nama	Kontribusi
13524026	Made Branenda Jordhy	
13524028	Muhammad Nur Majiid	
13524034	Jason Edward Salim	
13524068	Athilla Zaidan Zidna Fann	